Terraform, Airflow & Data Pipelines

Lecture 7 — How Terraform fits the stack, how Airflow can use Terraform, and the Lecture 6 assignment as a worked example.

---

1. Why Terraform works with Airflow and data pipelines

Data pipelines (e.g. Airflow DAGs) need **infrastructure** to run on: servers, containers, databases, object storage, message queues. Terraform is built to define and manage that infrastructure as code (IaC).

- **Same code → same environment** — Reproducible dev, staging, and production; no "works on my machine."
- **One tool for many providers** — AWS, GCP, Azure, Docker, Kubernetes; the same workflow can provision wherever Airflow and pipelines run.
- **Dependencies as code** — `depends_on` and resource ordering mirror DAG-style dependencies (run X before Y).
- **State and collaboration** — Remote state (e.g. S3 + DynamoDB lock) lets teams share one source of truth and avoid drift.
- **Variables and workspaces** — Different configs per environment without duplicating pipeline logic.

Bottom line: Terraform provisions and wires the "where" (servers, containers, queues, DBs); Airflow orchestrates the "when" and "what" (tasks and DAGs). They complement each other.

2. How Airflow can use Terraform

There are three main ways Airflow and Terraform work together.

2.1 Terraform provisions the environment Airflow runs on

Terraform creates and maintains the infrastructure that hosts Airflow and its dependencies. Examples:

- **Managed Airflow** — GCP Cloud Composer, AWS MWAA. Terraform can create the Composer environment or MWAA cluster, VPC, subnets, and IAM roles so the whole stack is reproducible.
- **Self-hosted on Kubernetes** — Terraform provisions the EKS/GKE/AKS cluster, node pools, and optionally the Helm release for the Airflow chart (e.g. `helm_release` in Terraform). Airflow then runs inside that cluster.

- **Self-hosted on VMs/containers** — Terraform creates EC2/ECS instances or a Docker Compose host, security groups, RDS (or other DB) for Airflow metadata, and S3/GCS buckets for DAGs and logs. `depends_on` ensures the database and buckets exist before the Airflow containers start.

In all cases, you run `terraform apply` (or a CI job that does it) when you change infrastructure; Airflow just runs on whatever Terraform has provisioned.

2.2 Terraform outputs feed into Airflow config

After Terraform creates resources, you often need their identifiers (ARNs, URLs, bucket names) inside Airflow. Two common patterns:

- **Terraform outputs → CI / config** — Your CI pipeline runs `terraform output -json` and writes the values into a config file or secret store that the Airflow deployment reads (e.g. env vars, Kubernetes ConfigMap). Airflow Variables or Connections can be seeded from that config.
- **Airflow Variables / Connections** — You can create a small script or Terraform run that, after `apply`, calls the Airflow REST API (or CLI) to set Variables and Connections using Terraform output values. That way DAGs reference `Variable.get("data_lake_bucket")` instead of hardcoding bucket names.

Example: Terraform creates an S3 bucket for raw data and outputs its name. A post-apply step sets Airflow Variable `raw_data_bucket` to that value. All DAGs that need the bucket use the variable, so they stay in sync with Terraform.

2.3 Running Terraform from inside Airflow (TerraformOperator)

Sometimes you want certain infrastructure changes to happen as part of a pipeline (e.g. "create a temporary cluster for this job, then destroy it"). Airflow can run Terraform from within a DAG:

- **TerraformOperator** — Community providers such as `airflow-terraform` (or custom operators) let you run `terraform init`, `plan`, and `apply` (or `destroy`) as Airflow tasks. The DAG can pass parameters (e.g. workspace, TF_VAR_*) and use task dependencies so "create infra" runs before "run job" and "destroy infra" runs after.
- **BashOperator / DockerOperator** — Alternatively, a task can run a script that invokes Terraform CLI in a container or on a worker that has Terraform installed. Less integrated but flexible.

Use this when infra is short-lived or environment-specific and you want it tied to the pipeline lifecycle. For long-lived, shared infrastructure (e.g. the Airflow cluster itself, central data lake), it's usually better to manage Terraform outside Airflow (CI/CD or manual runs) and only reference the resulting resources from DAGs.

> **Summary:** (1) Terraform provisions the Airflow runtime and its dependencies. (2) Terraform outputs can drive Airflow Variables/Connections so DAGs stay aligned with infra. (3)

> Optionally, Airflow can run Terraform via an operator or script for pipeline-scoped infra. Use (1) and (2) for most production setups; (3) for special cases.

3. Terraform in the data pipeline stack

Terraform (infrastructure): Compute (EC2, ECS, GKE, etc.), networking, security groups, IAM, databases, object storage, message queues — and the Docker/K8s environment where Airflow runs.

Airflow (orchestration): DAGs, task scheduling, retries, alerts, and dependencies. It runs on and uses the resources Terraform provisions.

Typical workflow: use Terraform to create the environment once (or when you change it); use Airflow to run pipelines on that environment repeatedly.

4. Lecture 6 assignment — case summary

Goal: Deploy a first webserver locally using Terraform.

What we built: A Docker container running **nginx**, serving a simple HTML page on **localhost:8080**.

Ingredients:

- An nginx image (e.g. `nginx:alpine`) pulled via the Docker provider.
- A small HTML file generated by Terraform (`local_file`) so content is in code.
- A container that mounts that HTML into nginx's docroot and exposes port 8080 on the host.

Order matters: The HTML file must exist before the container starts, so the container can mount it. Terraform expresses this with `depends_on = [local_file.index_html]` on the container.

Why it's a good example for pipelines: The same pattern — declare resources, set dependencies, one `apply` — scales to "run a webserver, then n8n" (Lecture 7) or "run a DB and buckets, then Airflow" in production.

5. Lecture 6 — how it's done with Terraform

Three main pieces in the assignment:

5.1 Providers and image

```
terraform {
  required_providers {
    docker = { source = "kreuzwerker/docker", version = "~> 3.0" }
    local  = { source = "hashicorp/local", version = "~> 2.0" }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name = "nginx:alpine"
}
```

5.2 Content and container

```
resource "local_file" "index_html" {
  content  = "<!DOCTYPE html><html>...</html>"
  filename = "${path.module}/index.html"
}
```

```
resource "docker_container" "webserver" {
  image     = docker_image.nginx.image_id
  name      = "terraform-assignment-webserver"
  must_run  = true
  depends_on = [local_file.index_html]

  ports {
    internal = 80
    external = 8080
  }

  volumes {
    host_path      = abspath(path.module)
    container_path = "/usr/share/nginx/html"
    read_only      = true
  }
}
```

5.3 Outputs and workflow

```
output "local_url" {
  value       = "http://localhost:8080"
  description = "URL to access your deployed HTML page (local)"
}

output "container_name" {
  value       = docker_container.webserver.name
  description = "Docker container name"
}
```

Commands: `terraform init` → `terraform plan` → `terraform apply`. To tear down: `terraform destroy`.

Takeaway: One declarative config (providers, resources, dependencies, outputs) gives you a reproducible local webserver. The same idea applies when provisioning the infrastructure for Airflow and data pipelines.

6. From Lecture 6 to Lecture 7 and production

- **Lecture 6:** One webserver (nginx + local HTML) with `depends_on` so the file exists before the container.
- **Lecture 7:** Webserver then n8n — same pattern: `depends_on = [docker_container.webserver]` on the n8n container so dependencies start first.
- **With Airflow:** Terraform can provision the cluster/VM/containers, database, and queues; Airflow DAGs then run on that infrastructure. Dependency order in Terraform (e.g. DB and buckets before Airflow) matches “infra ready before pipelines.”

Terraform works with Airflow and data pipelines because it owns the “where” and “in what order” the underlying resources come up. You use the same patterns — providers, resources, `depends_on`, outputs — from a simple local webserver (Lecture 6) all the way to production pipeline infrastructure.

Lecture 7 — Terraform, Airflow & Data Pipelines. Reference: Lecture 6 assignment, Lecture 7 examples and assignment (Docker/AWS).